

Học sâu và ứng dụng Mạng nơ-ron tích chập (Convolutional Neural Network)

Giảng viên: TS. Phạm Đình Phong

Email: phongpd@utc.edu.vn

Phone: 0972481813

Giới thiệu về xử lý ảnh

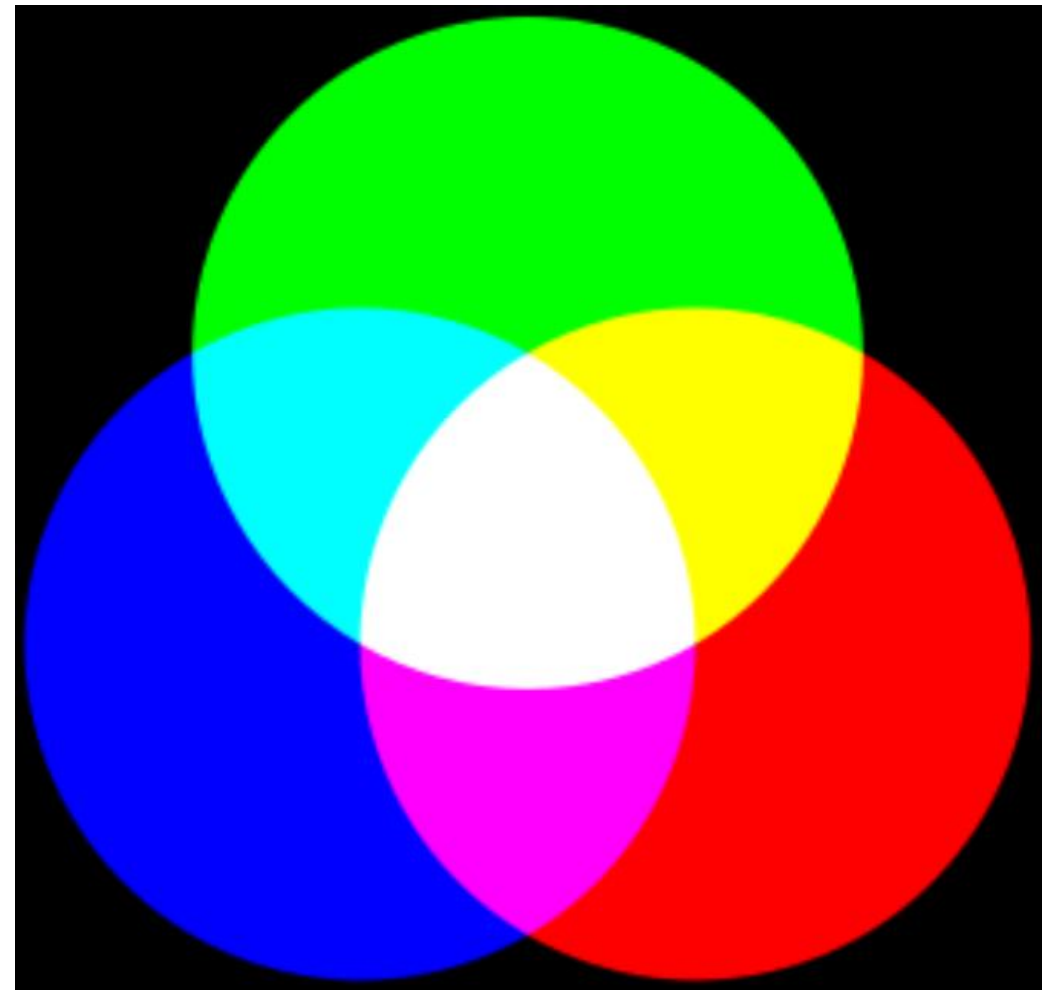
- Ảnh trong máy tính

- Hệ màu RGB

RGB viết tắt của red (đỏ), green (xanh lục), blue (xanh lam), là ba màu chính của ánh sáng khi tách ra từ lăng kính.

Khi trộn ba màu trên theo tỉ lệ nhất định có thể tạo thành các màu khác nhau.

Với mỗi bộ 3 số **r**, **g**, **b** nguyên trong khoảng $[0, 255]$ sẽ cho ra một màu khác nhau. Do có 256 cách chọn **r**, 256 cách chọn màu **g**, 256 cách chọn **b** $\Rightarrow$ tổng số màu có thể tạo ra bằng hệ màu RGB là:
 $256 * 256 * 256 = 16777216$ màu !!!



Thêm đỏ vào xanh lá cây tạo ra vàng,
thêm vàng vào xanh lam tạo ra trắng

Giới thiệu về xử lý ảnh

- Ảnh trong máy tính
 - Ảnh màu



Mathematical bridge, Cambridge

Giới thiệu về xử lý ảnh

- Ảnh trong máy tính

- Ảnh màu

- pixel (hay điểm ảnh) là một khối màu rất nhỏ và là đơn vị cơ bản nhất để tạo nên một bức ảnh kỹ thuật số.
 - Một bức ảnh kích thước 800 pixel * 600 pixel (chiều rộng * chiều cao), có thể được biểu diễn dưới dạng một ma trận kích thước 600 * 800 (vì định nghĩa ma trận là số hàng nhân số cột).

$$\begin{bmatrix} w_{1,1} & w_{1,2} & \dots & w_{1,800} \\ w_{2,1} & w_{2,2} & \dots & w_{2,800} \\ \dots & \dots & \dots & \dots \\ w_{600,1} & w_{600,2} & \dots & w_{600,800} \end{bmatrix}$$

trong đó mỗi phần tử $w_{i,j}$ là một pixel.

Giới thiệu về xử lý ảnh

- Ảnh trong máy tính

- Ảnh màu

- Có thể hiểu là mỗi pixel biểu diễn một màu và một bức ảnh là sự kết hợp rất nhiều pixel.
 - Khi in bức ảnh ra, kẻ ô vuông như chơi cờ ca rô với 800 đường thẳng ở chiều dài, 600 đường ở chiều rộng, thì mỗi ô vuông là một pixel, biểu diễn một chấm màu.
 - Tuy nhiên, để biểu diễn 1 màu ta cần 3 thông số (r, g, b) nên gọi $w_{i,j} = (r_{ij}, g_{ij}, b_{ij})$ và để biểu diễn dưới dạng ma trận thì sẽ như sau:

$$\begin{bmatrix} (100, 100, 50) & (101, 112, 3) & (131, 20, 80) \\ (150, 210, 130) & (10, 120, 130) & (111, 120, 130) \\ (10, 260, 30) & (200, 20, 30) & (100, 20, 3) \end{bmatrix}$$

Ảnh màu kích thước 3*3 biểu diễn dạng ma trận, mỗi pixel biểu diễn giá trị (r, g, b)

Giới thiệu về xử lý ảnh

- Ảnh trong máy tính
 - Ảnh màu
 - Để tiện lưu trữ và xử lý ảnh, tách mỗi giá trị trong mỗi pixel ra một ma trận riêng.

$$\begin{bmatrix} 100 & 101 & 131 \\ 150 & 10 & 111 \\ 10 & 200 & 100 \end{bmatrix}, \begin{bmatrix} 100 & 112 & 20 \\ 210 & 120 & 120 \\ 260 & 20 & 20 \end{bmatrix}, \begin{bmatrix} 50 & 3 & 80 \\ 130 & 130 & 130 \\ 30 & 30 & 3 \end{bmatrix}$$

R G B

Tách ma trận trên thành 3 ma trận cùng kích thước: mỗi ma trận lưu giá trị từng màu khác nhau red, green, blue.

Giới thiệu về xử lý ảnh

- Ảnh trong máy tính
 - Ảnh màu
 - Tổng quát

$$\begin{bmatrix} (r_{1,1}, g_{1,1}, b_{1,1}) & (r_{1,2}, g_{1,2}, b_{1,2}) & \dots & (r_{1,800}, g_{1,800}, b_{1,800}) \\ (r_{2,1}, g_{2,1}, b_{2,1}) & (r_{2,2}, g_{2,2}, b_{2,2}) & \dots & (r_{2,800}, g_{2,800}, b_{2,800}) \\ \dots & \dots & \dots & \dots \\ (r_{600,1}, g_{600,1}, b_{600,1}) & (r_{600,2}, g_{600,2}, b_{600,2}) & \dots & (r_{600,800}, g_{600,800}, b_{600,800}) \end{bmatrix}$$
$$\Rightarrow \begin{bmatrix} r_{1,1} & r_{1,2} & \dots & r_{1,800} \\ r_{2,1} & r_{2,2} & \dots & r_{2,800} \\ \dots & \dots & \dots & \dots \\ r_{600,1} & r_{600,2} & \dots & r_{600,800} \end{bmatrix}, \begin{bmatrix} g_{1,1} & g_{1,2} & \dots & g_{1,800} \\ g_{2,1} & g_{2,2} & \dots & g_{2,800} \\ \dots & \dots & \dots & \dots \\ g_{600,1} & g_{600,2} & \dots & g_{600,800} \end{bmatrix}, \begin{bmatrix} b_{1,1} & b_{1,2} & \dots & b_{1,800} \\ b_{2,1} & b_{2,2} & \dots & b_{2,800} \\ \dots & \dots & \dots & \dots \\ b_{600,1} & b_{600,2} & \dots & b_{600,800} \end{bmatrix}$$

Mỗi ma trận được tách ra được gọi là 1 channel nên ảnh màu được gọi là 3 channel: channel red, channel green, channel blue.

Giới thiệu về xử lý ảnh

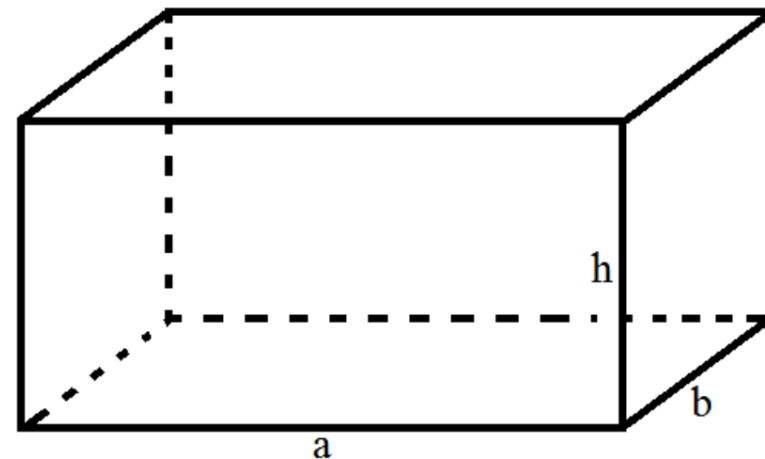
- Ảnh trong máy tính
 - Tensor là gì
 - Khi dữ liệu được biểu diễn dạng 1 chiều, người ta gọi là vector, mặc định khi viết vector sẽ viết dưới dạng cột.
 - Khi dữ liệu dạng 2 chiều, người ta gọi là ma trận, kích thước là số hàng * số cột.
 - Khi dữ liệu nhiều hơn 2 chiều thì sẽ được gọi là tensor.
 - Ta thấy rằng ma trận là sự kết hợp của các vector cùng kích thước. Xếp n vector kích thước m cạnh nhau thì sẽ được ma trận $m \times n$.
 - Tensor 3 chiều là sự kết hợp của các ma trận cùng kích thước, xếp k ma trận kích thước $m \times n$ lên nhau sẽ được tensor kích thước $m \times n \times k$.

Giới thiệu về xử lý ảnh

- Ảnh trong máy tính

- Tensor là gì

- Tưởng tượng mặt đáy là một ma trận kích thước $a * b$, được tạo bởi b vector kích thước a . Cả hình hộp là tensor 3 chiều kích thước $a*b*h$, được tạo bởi xếp h ma trận kích thước $a*b$ lên nhau.
 - Do đó, ảnh màu trên máy tính sẽ được biểu diễn dưới dạng tensor 3 chiều kích thước $height*width*3$ (ví dụ: $600*800*3$) do có 3 ma trận (channel) màu red, green, blue kích thước $height*width$ ($600*800$) chồng lên nhau.

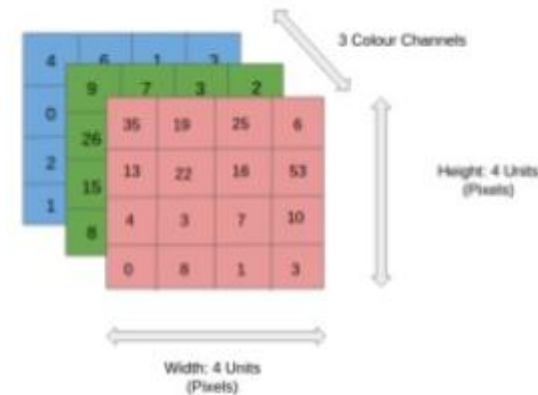
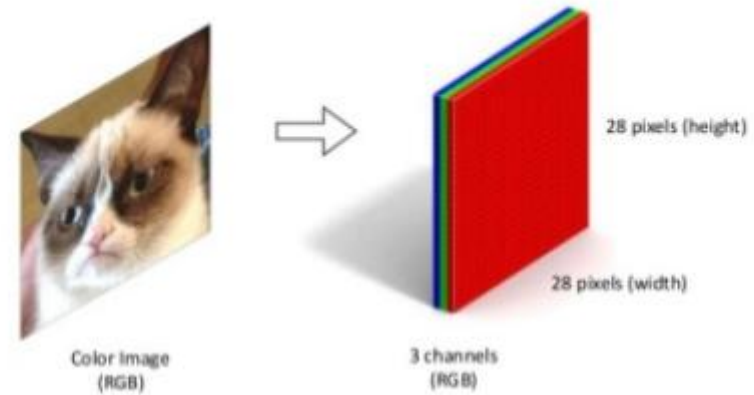


Giới thiệu về xử lý ảnh

- Ảnh trong máy tính
 - Tensor là gì

Ví dụ biểu diễn ảnh màu kích thước 28×28 , biểu diễn dưới dạng tensor $28 \times 28 \times 3$.

color image is 3rd-order tensor



Giới thiệu về xử lý ảnh

- Ảnh trong máy tính
 - Ảnh xám

Ảnh xám của mathematical bridge



Giới thiệu về xử lý ảnh

- Ảnh trong máy tính
 - Ảnh xám
 - Tương tự ảnh màu, ảnh xám có kích thước 800 pixel * 600 pixel có thể được biểu diễn dưới dạng một ma trận kích thước 600 * 800 (vì định nghĩa ma trận là số hàng nhân số cột).

$$\begin{bmatrix} w_{1,1} & w_{1,2} & \dots & w_{1,800} \\ w_{2,1} & w_{2,2} & \dots & w_{2,800} \\ \dots & \dots & \dots & \dots \\ w_{600,1} & w_{600,2} & \dots & w_{600,800} \end{bmatrix}$$

trong đó mỗi phần tử $w_{i,j}$ là một pixel.

Giới thiệu về xử lý ảnh

- Ảnh trong máy tính
 - Ảnh xám
 - Mỗi pixel trong ảnh xám chỉ cần biểu diễn bằng một giá trị nguyên trong khoảng từ $[0, 255]$ thay vì (r,g,b) như trong ảnh màu. Do đó, khi biểu diễn ảnh xám trong máy tính chỉ cần một ma trận là đủ.

Biểu diễn ảnh xám

$$\begin{bmatrix} 0 & 215 & \dots & 250 \\ 12 & 156 & \dots & 1 \\ \dots & \dots & \dots & \dots \\ 244 & 255 & \dots & 12 \end{bmatrix}$$

- Giá trị 0 là màu đen, 255 là màu trắng và giá trị pixel càng gần 0 thì càng tối và càng gần 255 thì càng sáng.

Giới thiệu về xử lý ảnh

- Ảnh trong máy tính
 - Chuyển hệ màu của ảnh
 - Mỗi pixel trong ảnh màu được biểu diễn bằng 3 giá trị (r, g, b) còn trong ảnh xám chỉ cần 1 giá trị x để biểu diễn.
 - Khi chuyển từ ảnh màu sang ảnh xám ta có thể dùng công thức:
$$x = r * 0.299 + g * 0.587 + b * 0.114.$$
 - Tuy nhiên, khi chuyển ngược lại, ta chỉ biết giá trị x và cần đi tìm r, g, b nên sẽ không chính xác.

Giới thiệu về xử lý ảnh

- Phép tính convolution
 - Convolution (tích chập)
 - Để cho dễ hình dung, ta sẽ lấy ví dụ trên ảnh xám, tức là ảnh được biểu diễn dưới dạng ma trận A kích thước $m \times n$.
 - Ta định nghĩa kernel là một ma trận vuông kích thước $k \times k$ trong đó k là số lẻ. k có thể bằng 1, 3, 5, 7, 9, ... Ví dụ kernel kích thước 3×3 .

$$W = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

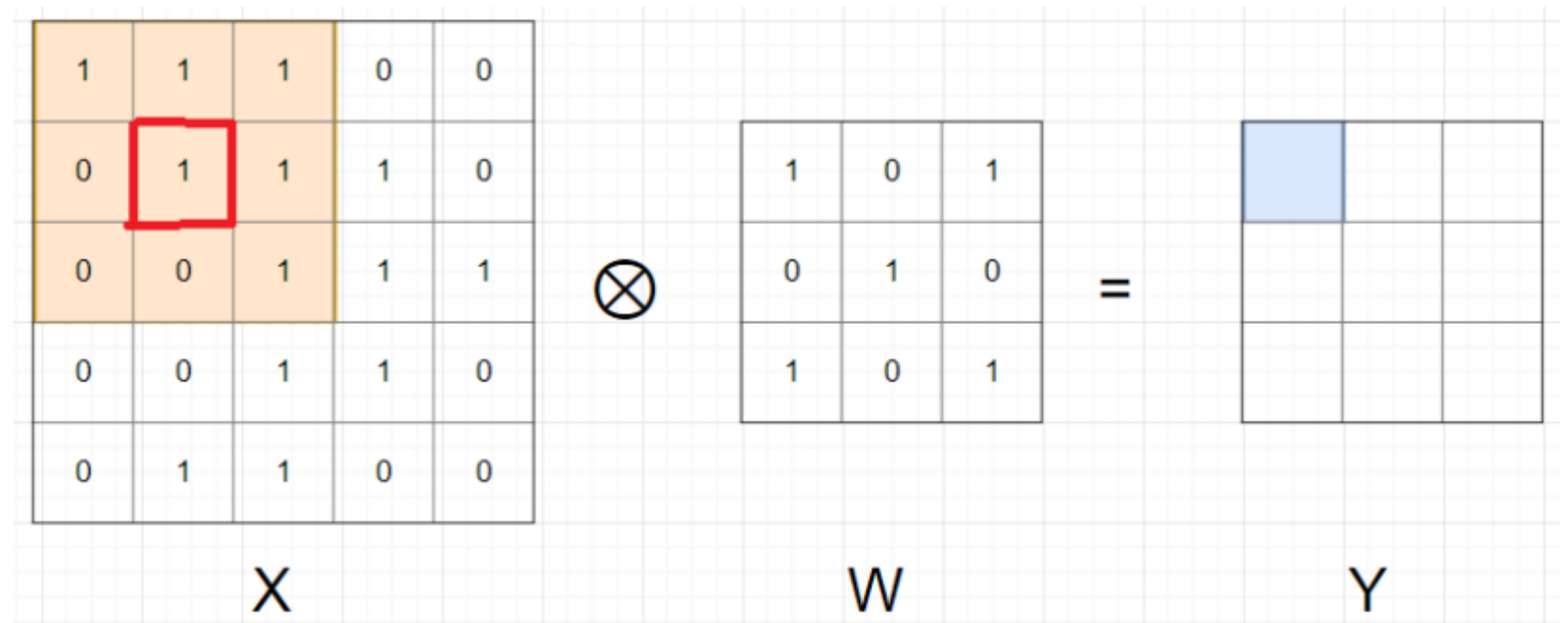
- Kí hiệu phép tính convolution ($\otimes$), kí hiệu $Y = X \otimes W$

Giới thiệu về xử lý ảnh

- Phép tính convolution
 - Convolution (tích chập)
 - Với mỗi phần tử x_{ij} trong ma trận X lấy ra một ma trận có kích thước bằng kích thước của kernel W có phần tử x_{ij} làm trung tâm (lý do vì sao kích thước của kernel thường lẻ) gọi là ma trận A .
 - Sau đó tính tổng các phần tử của phép tính element-wise của ma trận A và ma trận W , rồi viết vào ma trận kết quả Y .

Giới thiệu về xử lý ảnh

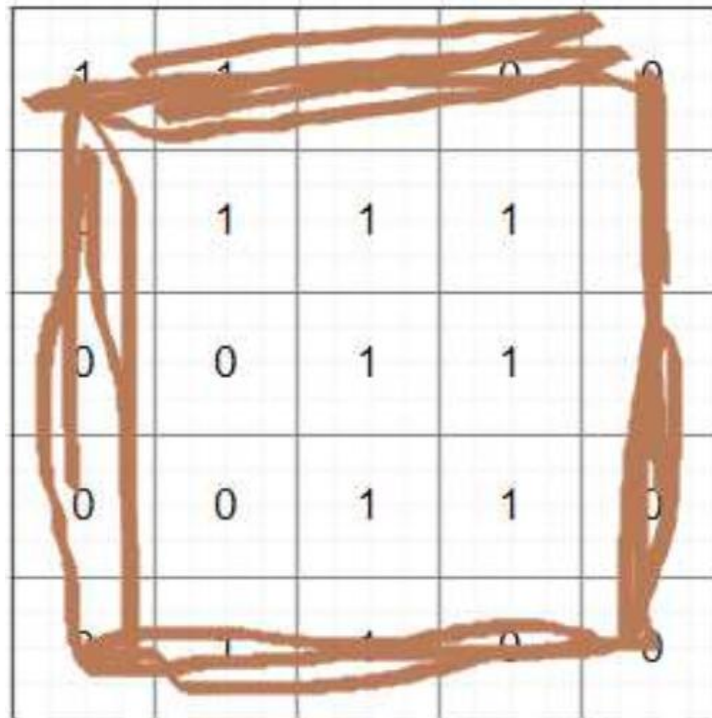
- Phép tính convolution
 - Convolution (tích chập)



Ví dụ khi tính tại x_{22} (ô khoanh đỏ trong hình), ma trận A cùng kích thước với W , có x_{22} làm trung tâm có màu nền da cam như trong hình. Sau đó tính $y_{11} = \text{sum}(A \otimes W) = x_{11} * w_{11} + x_{12} * w_{12} + x_{13} * w_{13} + x_{21} * w_{21} + x_{22} * w_{22} + x_{23} * w_{23} + x_{31} * w_{31} + x_{32} * w_{32} + x_{33} * w_{33} = 4$. Và làm tương tự với các phần tử còn lại trong ma trận.

Giới thiệu về xử lý ảnh

- Phép tính convolution
 - Convolution (tích chập)
 - Thế thì sẽ xử lý thế nào với phần tử ở viền ngoài như x_{11} ? Bình thường khi tính thì sẽ bỏ qua các phần tử ở viền ngoài, vì không tìm được ma trận A ở trong X.



Giới thiệu về xử lý ảnh

- Phép tính convolution
 - Convolution (tích chập)
 - Để ý ta thấy ma trận Y có kích thước nhỏ hơn ma trận X. Kích thước của ma trận Y là $(m-k+1) * (n-k+1)$.

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

Giới thiệu về xử lý ảnh

- Phép tính convolution
 - Padding
 - Như ở trên thì mỗi lần thực hiện phép tính convolution xong thì kích thước ma trận Y đều nhỏ hơn X.
 - Giờ ta muốn ma trận Y thu được có kích thước bằng ma trận X
 - => Tìm cách giải quyết cho các phần tử ở viền
 - => Thêm giá trị 0 ở viền ngoài ma trận X.

Giới thiệu về xử lý ảnh

- Phép tính convolution
 - Padding

Rõ ràng là giờ đã giải quyết được vấn đề tìm A cho phần tử x_{11} , và ma trận Y thu được sẽ bằng kích thước ma trận X ban đầu.

Phép tính này gọi là convolution với padding=1. Padding=k nghĩa là thêm k vector 0 vào mỗi phía của ma trận.

0	0	0	0	0	0	0
0	1	1	1	0	0	0
0	0	1	1	1	0	0
0	0	0	1	1	1	0
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0

Ma trận X khi thêm viền 0 bên ngoài

Giới thiệu về xử lý ảnh

- Phép tính convolution
 - Stride (bước nhảy)

Như ở trên ta thực hiện tuần tự các phần tử trong ma trận X, thu được ma trận Y cùng kích thước ma trận X, ta gọi là stride=1.

Tuy nhiên, nếu stride=k ($k > 1$) thì ta chỉ thực hiện phép tính convolution trên các phần tử

$$X_{1+i*k, 1+j*k}$$

0	0	0	0	0	0	0
0	1	1	1	0	0	0
0	0	1	1	1	0	0
0	0	0	1	1	1	0
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0

stride=1, padding=1

Giới thiệu về xử lý ảnh

- Phép tính convolution
 - Stride

Ví dụ $k = 2$ như ở hình bên.

Hiểu đơn giản là bắt đầu từ vị trí x_{11} sau đó nhảy k bước theo chiều dọc và ngang cho đến hết ma trận X .

Kích thước của ma trận Y là 3×3 đã giảm đi đáng kể so với ma trận X .

0	0	0	0	0	0	0
0	1	1	1	0	0	0
0	0	1	1	1	0	0
0	0	0	1	1	1	0
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0

padding=1, stride=2

Giới thiệu về xử lý ảnh

- Phép tính convolution

- Stride

- Công thức tổng quát cho phép tính convolution của ma trận X kích thước $m \times n$ với kernel kích thước $k \times k$, stride = s , padding = p ra ma trận Y kích thước

$$\left(\frac{m-k+2*p}{s} + 1 \right) * \left(\frac{n-k+2*p}{s} + 1 \right).$$

- Stride thường dùng để giảm kích thước của ma trận sau phép tính convolution.

Giới thiệu về xử lý ảnh

- Phép tính convolution
 - Ý nghĩa của phép tính convolution
 - Mục đích của phép tính convolution trên ảnh là làm mờ, làm nét ảnh, xác định các đường, ...
 - Mỗi kernel khác nhau thì phép tính convolution sẽ có ý nghĩa khác nhau.

Giới thiệu về xử lý ảnh

- Phép tính convolution

Operation	Kernel ω	Image result $g(x,y)$
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	

Giới thiệu về xử lý ảnh

- Phép tính convolution

Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	

Convolutional neural network - CNN

- Thiết lập bài toán
 - Gần đây việc kiểm tra mã captcha để xác minh không phải robot của Google bị chính robot vượt qua
 - Do đó, Google quyết định cho ra thuật toán mới, dùng camera chụp ảnh người dùng và dùng deep learning để xác minh xem ảnh có chứa mặt người không thay cho hệ thống captcha cũ.

Bài toán: Đầu vào (Input) là một ảnh màu kích thước 64×64 , đầu ra (output) ảnh có chứa mặt người hay không.

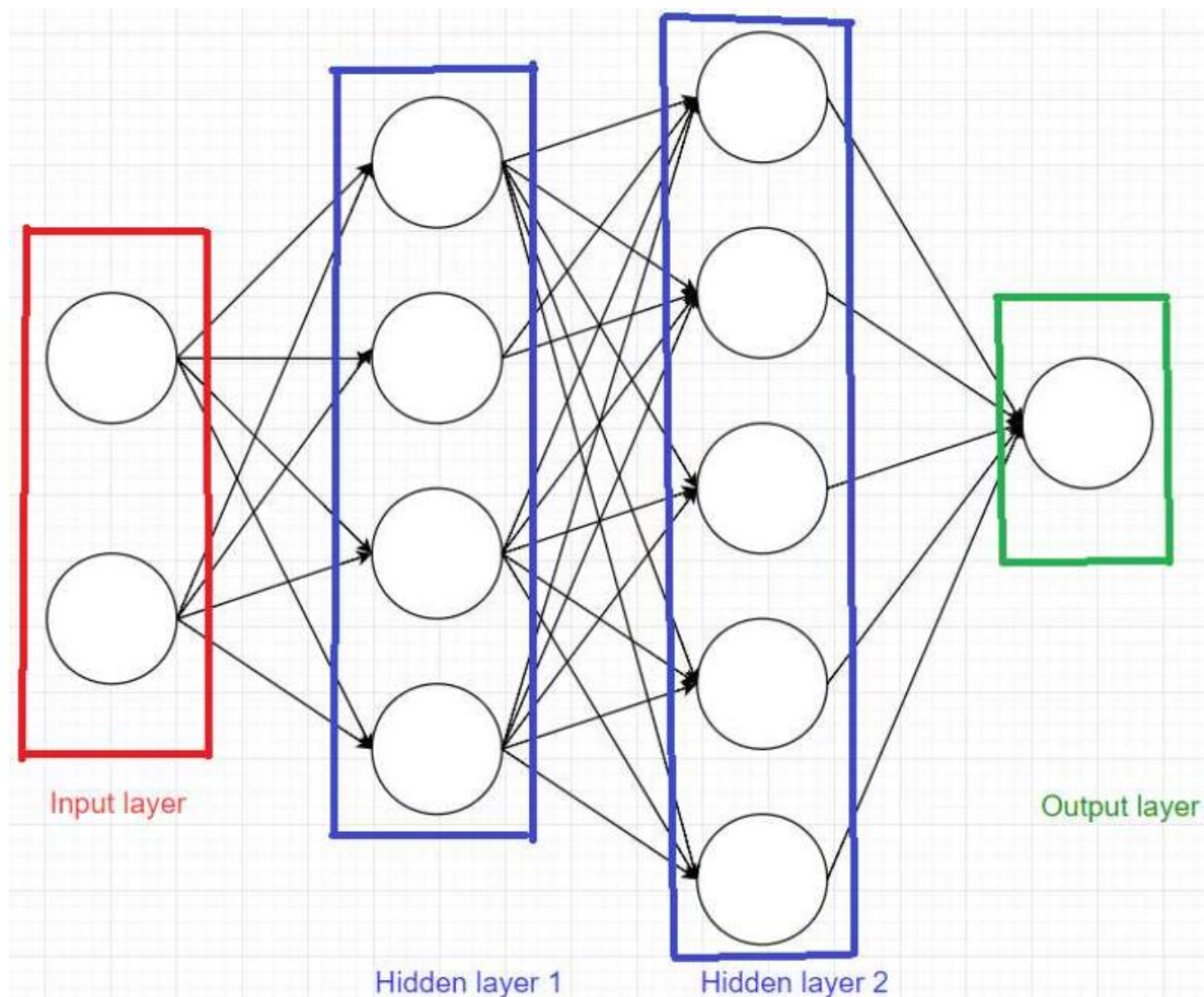
Convolutional neural network - CNN

- Convolutional layer

Trong mô hình neural network từ những bài trước:

Mỗi hidden layer được gọi là fully connected layer, tên gọi theo đúng ý nghĩa, mỗi node trong hidden layer được kết nối với tất cả các node trong layer trước.

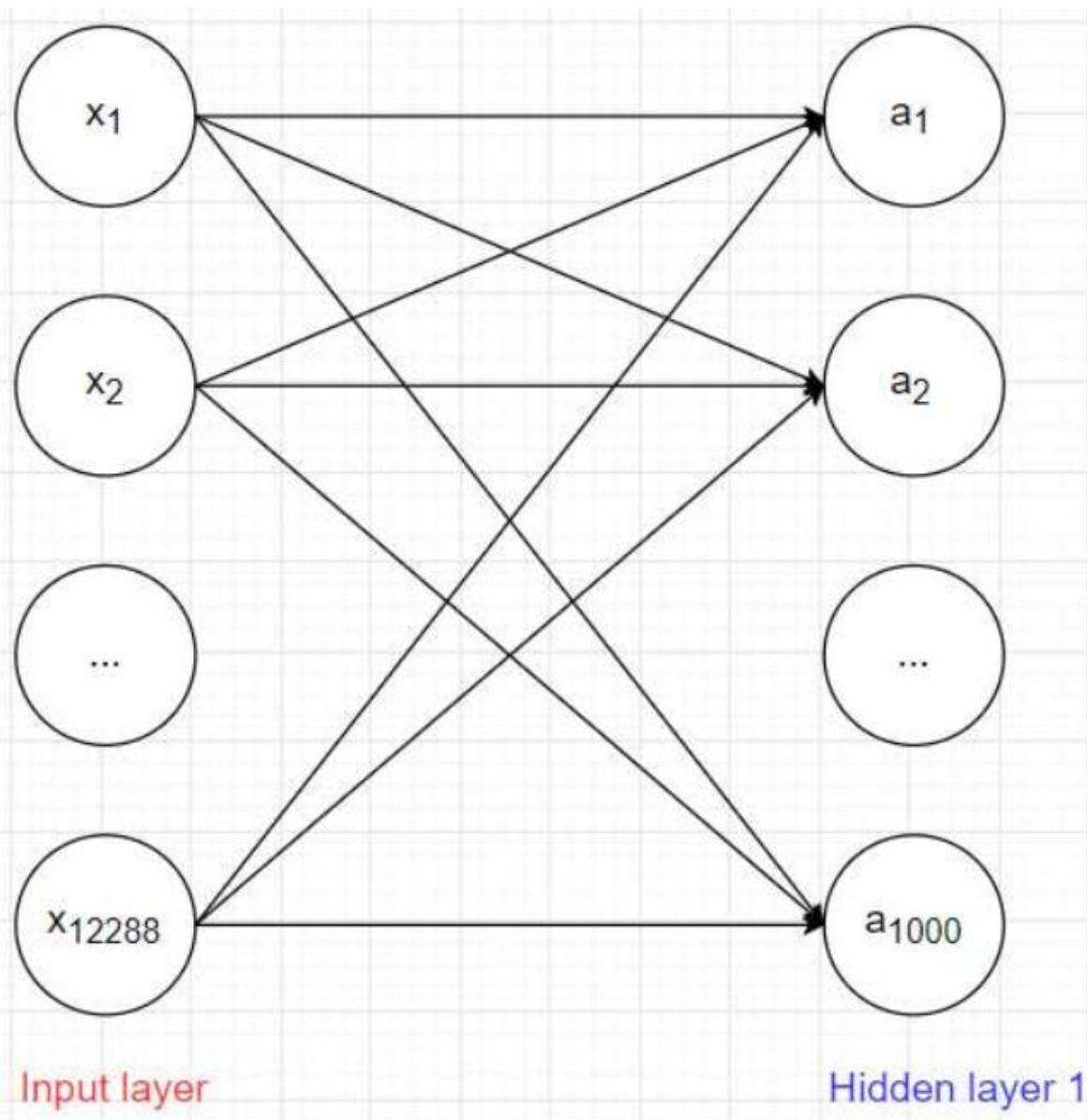
Cả mô hình được gọi là fully connected neural network (FCN).



Convolutional neural network - CNN

- Convolutional layer
 - Vấn đề của FCN với xử lý ảnh

Trong xử lý ảnh, ảnh màu 64×64 được biểu diễn dưới dạng 1 tensor $64 \times 64 \times 3$.
Nên để biểu thị hết nội dung của bức ảnh thì cần truyền vào input layer tất cả các pixel ($64 \times 64 \times 3 = 12288$). Nghĩa là input layer giờ có 12288 nodes.



Convolutional neural network - CNN

- Convolutional layer
 - Vấn đề của FCN với xử lý ảnh
 - Giả sử số lượng node trong hidden layer 1 là 1000. Số lượng weight W giữa input layer và hidden layer 1 là $12288 * 1000 = 12288000$, số lượng bias là 1000 $\Rightarrow$ tổng số parameter là: 12289000.
 - Ví dụ trên mới chỉ là số tham số (parameter) giữa input layer và hidden layer 1, trong model còn nhiều layer nữa và nếu kích thước ảnh tăng, ví dụ $512 * 512$, thì số lượng parameter tăng cực kì nhanh $\Rightarrow$ Cần giải pháp tốt hơn !!!

Convolutional neural network - CNN

- Convolutional layer

- Vấn đề của FCN với xử lý ảnh

- Nhận xét:

- Trong ảnh các pixel ở cạnh nhau thường có liên kết với nhau hơn là những pixel ở xa. Ví dụ như phép tính convolution trên ảnh ở bài trước. Để tìm các đường trong ảnh, ta áp dụng sobel kernel trên mỗi vùng kích thước 3×3 . Hay làm nét ảnh ta áp dụng sharpen kernel cũng trên vùng có kích thước 3×3 .
 - Trong phép tính convolution trong ảnh, chỉ 1 kernel được dùng trên toàn bộ bức ảnh. Nói cách khác là các pixel ảnh chia sẻ hệ số với nhau.

=> Áp dụng phép tính convolution vào layer trong neural network ta có thể giải quyết được vấn đề lượng lớn parameter mà vẫn lấy ra được các đặc trưng của ảnh.

Convolutional neural network - CNN

- Convolutional layer

- Convolutional layer đầu tiên

- Bài trước phép tính convolution thực hiện trên ảnh xám với biểu diễn ảnh dạng ma trận

Tuy nhiên, ảnh màu có tới 3 channels red, green, blue nên khi biểu diễn ảnh dưới dạng tensor 3 chiều ta cũng định nghĩa kernel là 1 tensor 3 chiều kích thước $k \times k \times 3$.

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

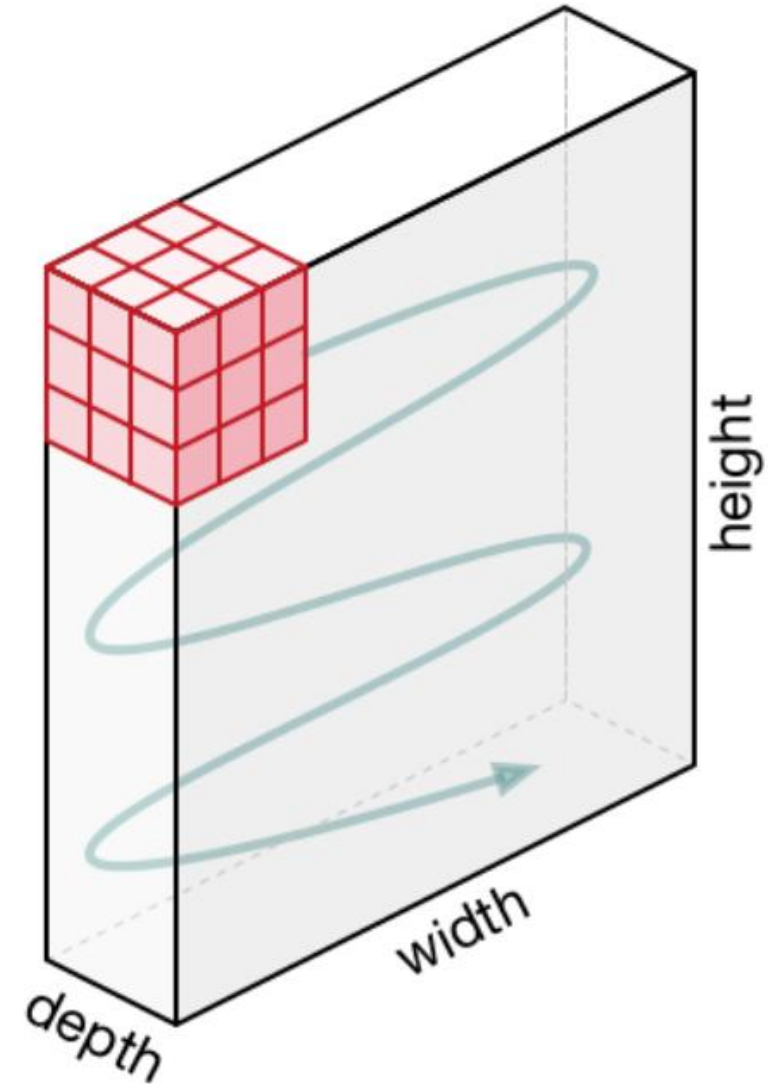
4		

Convolved
Feature

Convolutional neural network - CNN

- Convolutional layer
 - Convolutional layer đầu tiên

Ta định nghĩa kernel có cùng độ sâu (depth) với biểu diễn ảnh, rồi sau đó thực hiện di chuyển khối kernel tương tự như khi thực hiện trên ảnh xám.



Convolutional neural network - CNN

- Convolutional layer
 - Convolutional layer đầu tiên

Tensor X, W 3 chiều được viết dưới dạng 3 matrix

0	0	0	0	0	0
0	156	155	156	158	0
0	153	154	157	159	0
0	149	151	155	159	0
0	146	146	149	153	0
0	0	0	0	0	0

0	0	0	0	0	0
0	167	166	167	158	0
0	164	165	168	159	0
0	160	162	166	159	0
0	146	146	149	153	0
0	0	0	0	0	0

0	0	0	0	0	0
0	163	162	163	158	0
0	160	161	164	159	0
0	156	158	162	159	0
0	146	146	149	153	0
0	0	0	0	0	0

X

-1	-1	1
0	1	-1
0	1	1

1	0	0
1	-1	-1
1	0	-1

0	1	1
0	1	0
1	-1	1

W

$\otimes$

=

Y

Convolutional neural network - CNN

- Convolutional layer

- Convolutional layer đầu tiên

- Khi biểu diễn ma trận ta cần 2 chỉ số hàng i và cột j, khi biểu diễn ở dạng tensor 3 chiều cần thêm chỉ số độ sâu k nên chỉ số mỗi phần tử trong tensor là x_{ijk} .

$$y_{11} = b + (x_{111} * w_{111} + x_{121} * w_{121} + x_{131} * w_{131} + x_{211} * w_{211} + x_{221} * w_{221} + x_{231} * w_{231} + x_{311} * w_{311} + x_{321} * w_{321} + x_{331} * w_{331}) + (x_{112} * w_{112} + x_{122} * w_{122} + x_{132} * w_{132} + x_{212} * w_{212} + x_{222} * w_{222} + x_{232} * w_{232} + x_{312} * w_{312} + x_{322} * w_{322} + x_{332} * w_{332}) + (x_{113} * w_{113} + x_{123} * w_{123} + x_{133} * w_{133} + x_{213} * w_{213} + x_{223} * w_{223} + x_{233} * w_{233} + x_{313} * w_{313} + x_{323} * w_{323} + x_{333} * w_{333}) = -25$$

0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1



308

+

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2



-498

+

0	1	1
0	1	0
1	-1	1

Kernel Channel #3



164

+ 1 = -25



Bias = 1

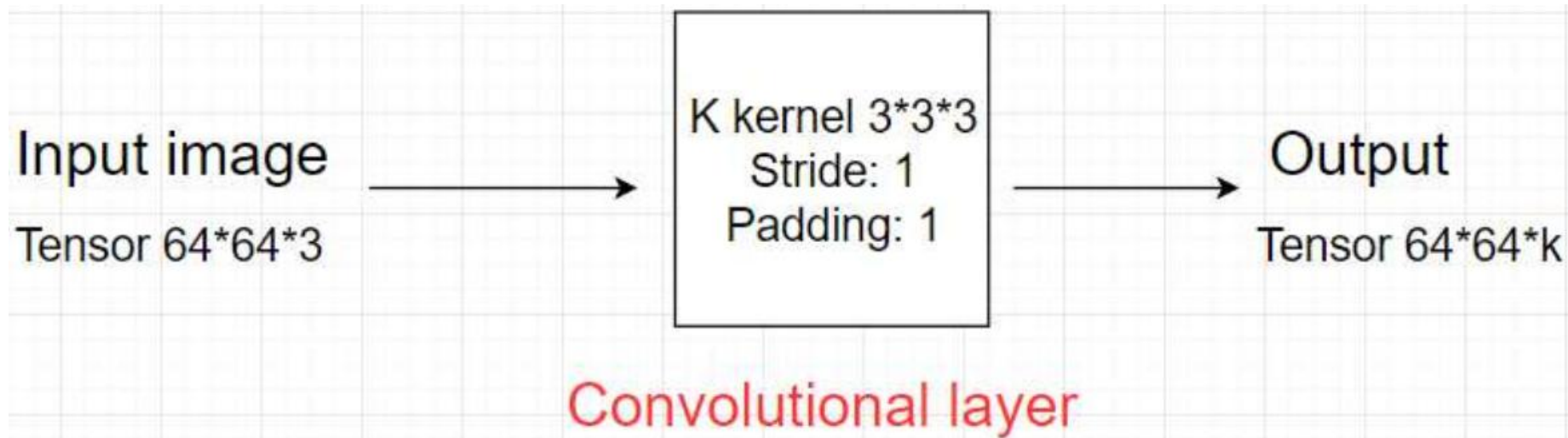
Output

-25				...
				...
				...
				...
...	...	...	...	...

Convolutional neural network - CNN

- Convolutional layer

- Với mỗi kernel khác nhau sẽ học được những đặc trưng khác nhau của ảnh nên trong mỗi convolutional layer ta sẽ dùng nhiều kernel để học được nhiều thuộc tính của ảnh. Vì mỗi kernel cho ra output là 1 matrix nên k kernel sẽ cho ra k output matrix. Ta kết hợp k output matrix này lại thành 1 tensor 3 chiều có chiều sâu k.



Output của convolutional layer đầu tiên sẽ thành input của convolutional layer tiếp theo.

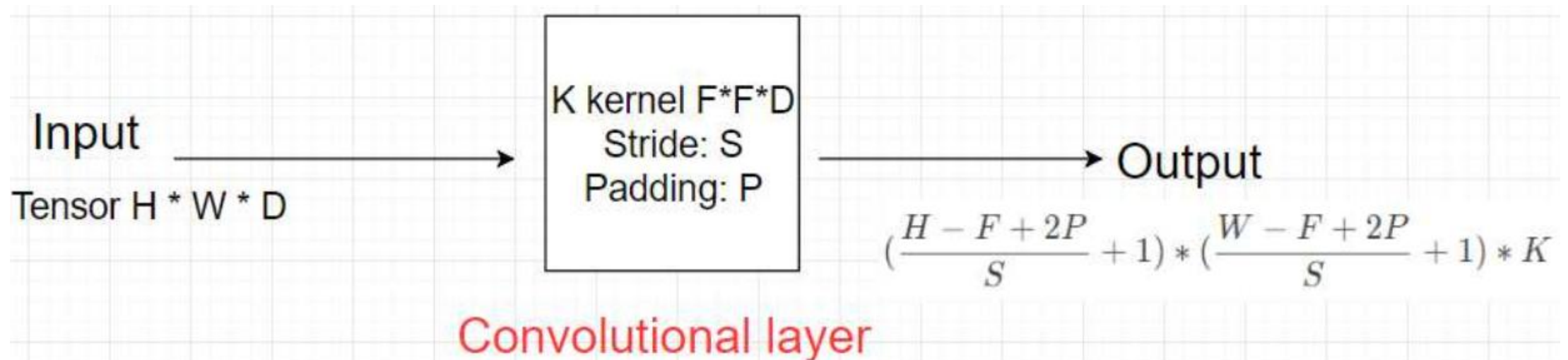
Convolutional neural network - CNN

- Convolutional layer tổng quát

- Giả sử input của 1 convolutional layer tổng quát là tensor kích thước $H*W*D$.
- Kernel có kích thước $F*F*D$ (kernel luôn có depth bằng depth của input và F là số lẻ), stride: S , padding: P .

=> Output của layer là tensor 3 chiều có kích thước:

$$\left(\frac{H - F + 2P}{S} + 1\right) * \left(\frac{W - F + 2P}{S} + 1\right) * K$$



Convolutional neural network - CNN

- Convolutional layer tổng quát

- Lưu ý:

- Output của convolutional layer sẽ qua hàm activation function trước khi trở thành input của convolutional layer tiếp theo.
- Tổng số parameter của layer: Mỗi kernel có kích thước $F \times F \times D$ và có 1 hệ số bias, nên tổng parameter của 1 kernel là $F \times F \times D + 1$. Mà convolutional layer áp dụng K kernel $\Rightarrow$ Tổng số parameter trong layer này là $K * (F \times F \times D + 1)$.

Convolutional neural network - CNN

- Pooling layer (tầng gộp)
 - Pooling layer thường được dùng giữa các convolutional layer, để giảm kích thước dữ liệu nhưng vẫn giữ được các thuộc tính quan trọng. Kích thước dữ liệu giảm giúp giảm việc tính toán trong model.
 - Gọi pooling size kích thước $K \times K$.
 - Input của pooling layer có kích thước $H \times W \times D$, được tách ra làm D ma trận kích thước $H \times W$.
 - Với mỗi ma trận, trên vùng kích thước $K \times K$ trên ma trận ta tìm maximum hoặc average của dữ liệu rồi viết vào ma trận kết quả. Quy tắc về stride và padding áp dụng như phép tính convolution trên ảnh.

Convolutional neural network - CNN

- Pooling layer (tầng gộp)

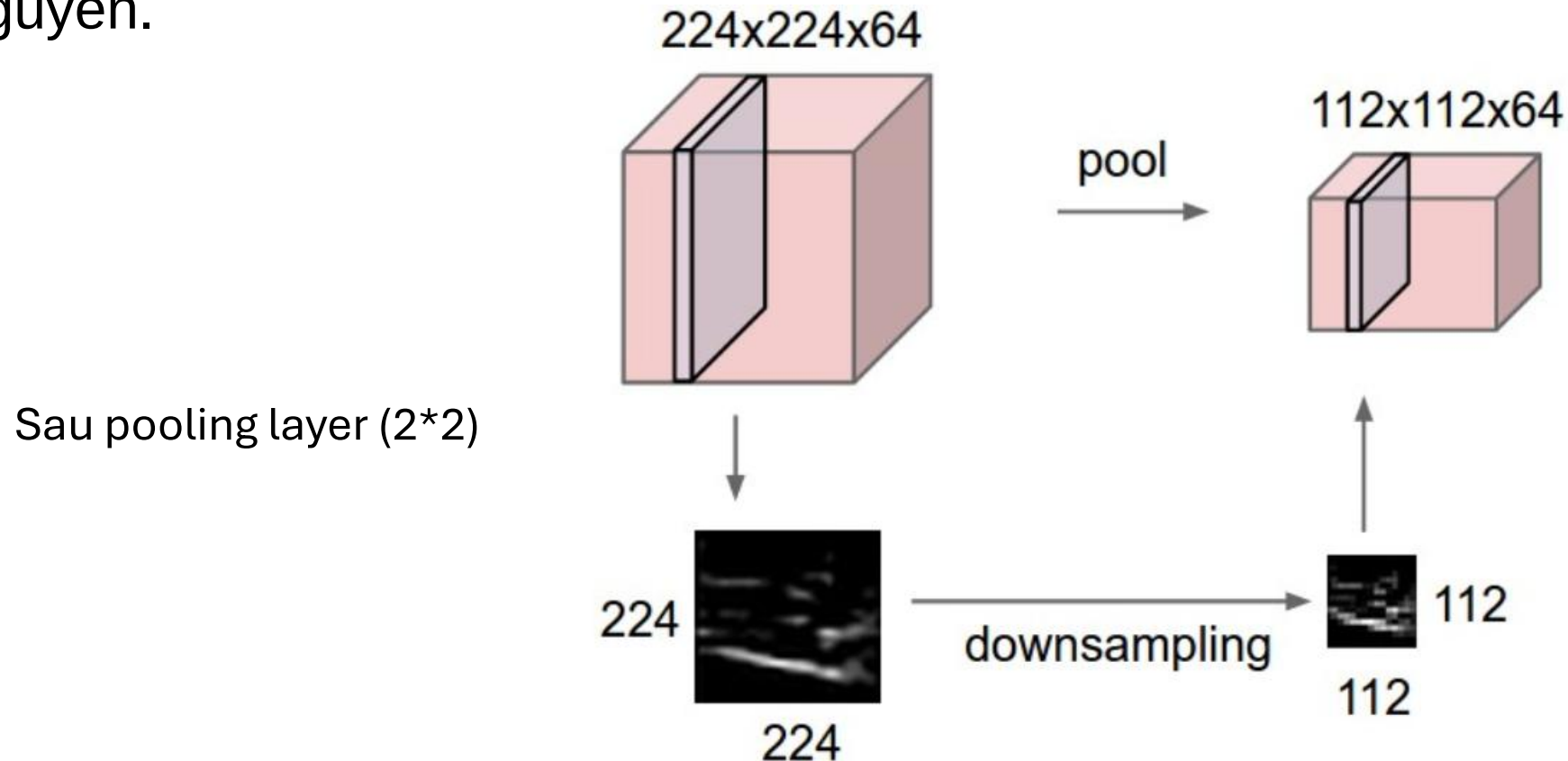
3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

max pooling layer với size=(3,3), stride=1, padding=0

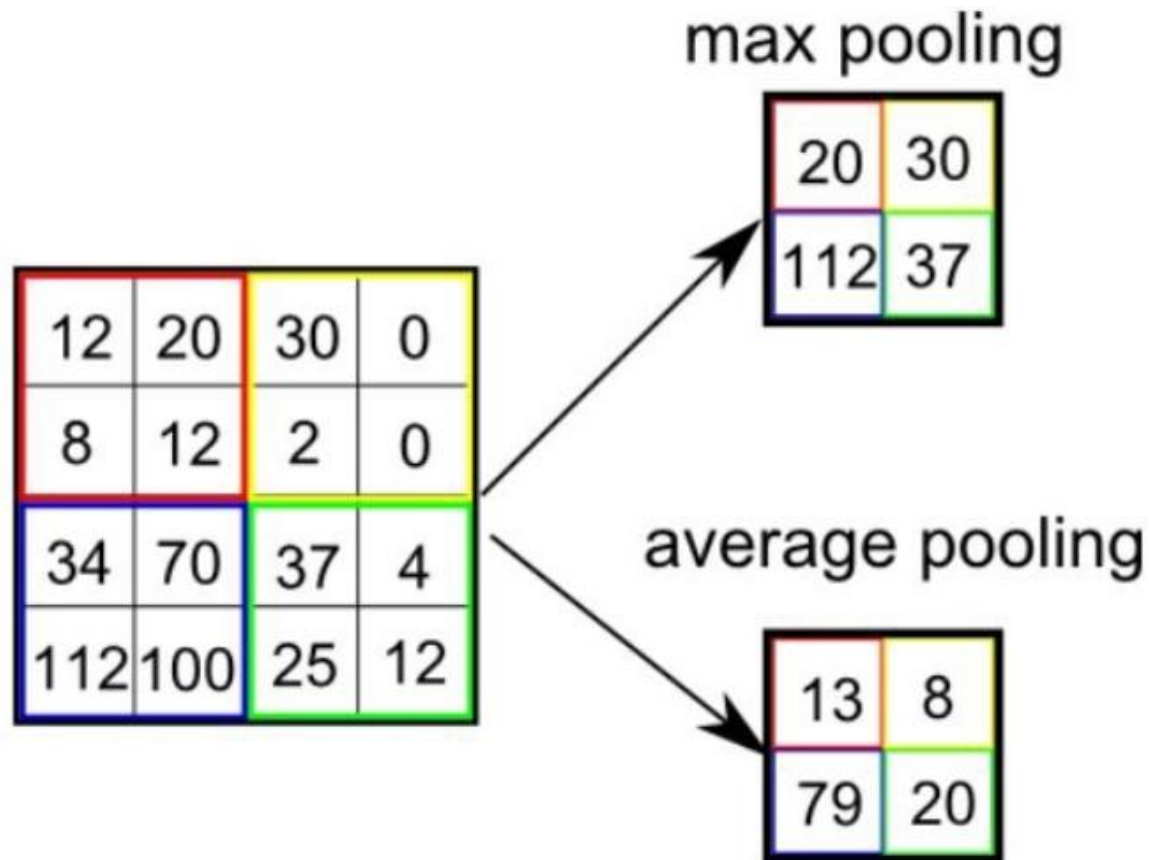
Convolutional neural network - CNN

- Pooling layer (tầng gộp)
 - Hầu hết khi dùng pooling layer thì sẽ dùng size=(2, 2), stride=2, padding=0. Khi đó output width và height của dữ liệu giảm đi một nửa, depth thì được giữ nguyên.



Convolutional neural network - CNN

- Pooling layer (tầng gộp)
 - Có 2 loại pooling layer phổ biến là: max pooling và average pooling.

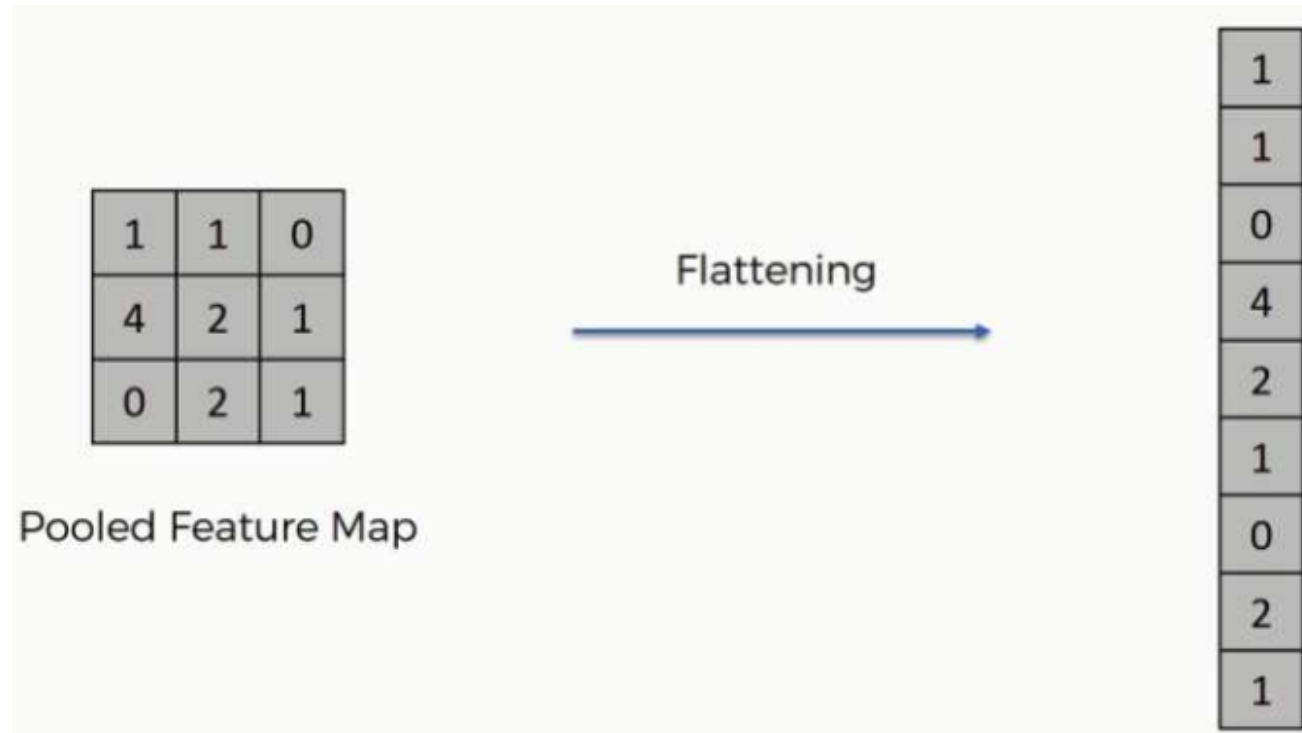


Trong một số model người ta dùng convolutional layer với stride > 1 để giảm kích thước dữ liệu thay cho pooling layer.

Convolutional neural network - CNN

- Fully connected layer

- Sau khi ảnh được truyền qua nhiều convolutional layer và pooling layer thì model đã học được tương đối các đặc điểm của ảnh (ví dụ mắt, mũi, khung mặt, ...) thì tensor của output của layer cuối cùng, kích thước $H*W*D$, sẽ được chuyển về 1 vector kích thước $(H*W*D)$

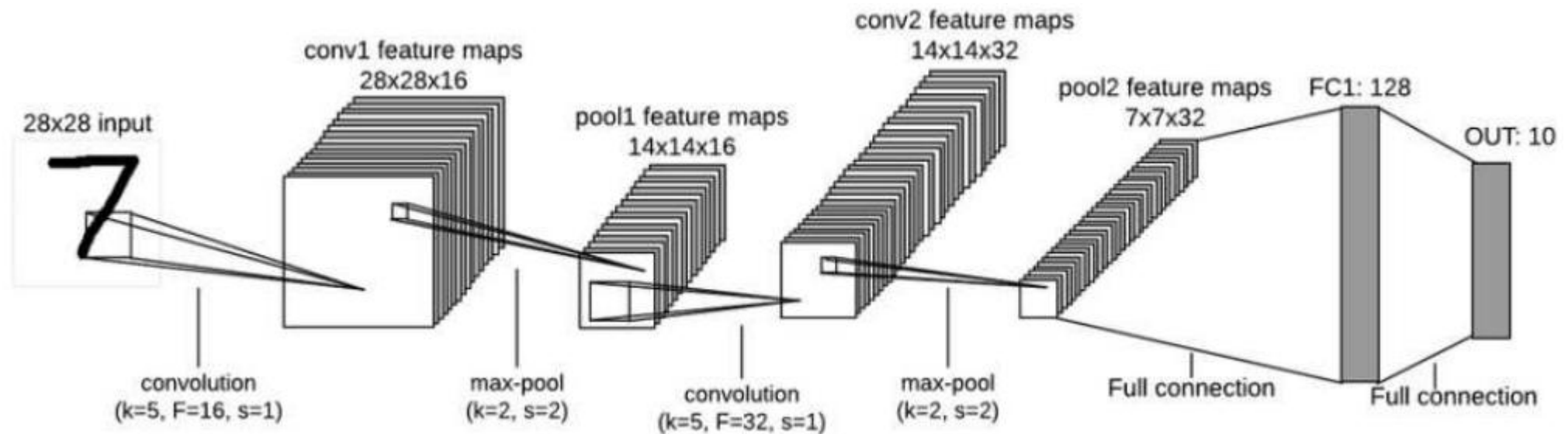


Sau đó ta dùng các fully connected layer để kết hợp các đặc điểm của ảnh để ra được output của model.

Convolutional neural network - CNN

- Visualise convolutional neural network
 - Mô hình convolutional neural network:

Input image -> Convolutional layer (Conv) + Pooling layer (Pool) -> Fully connected layer (FC) -> Output.



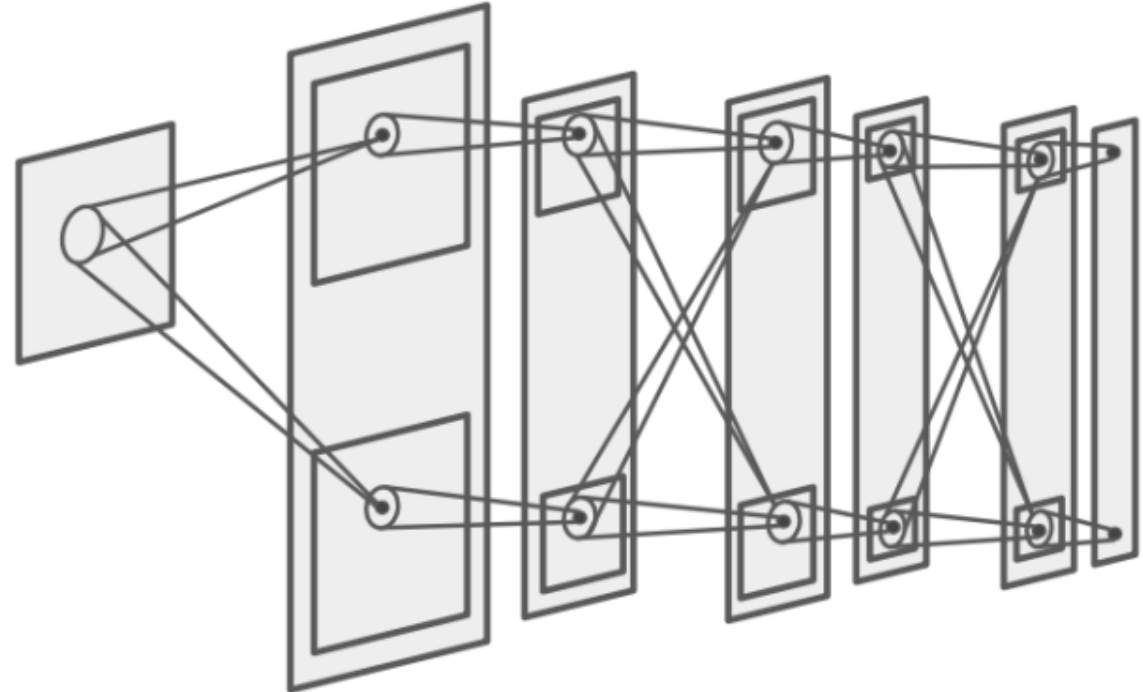
Ví dụ mô hình convolutional neural network

Convolutional neural network - CNN

- Lịch sử CNNs

Neocognitron *[Fukushima 1980]*

“sandwich” architecture (SCSCSC...)
simple cells: modifiable parameters
complex cells: perform pooling



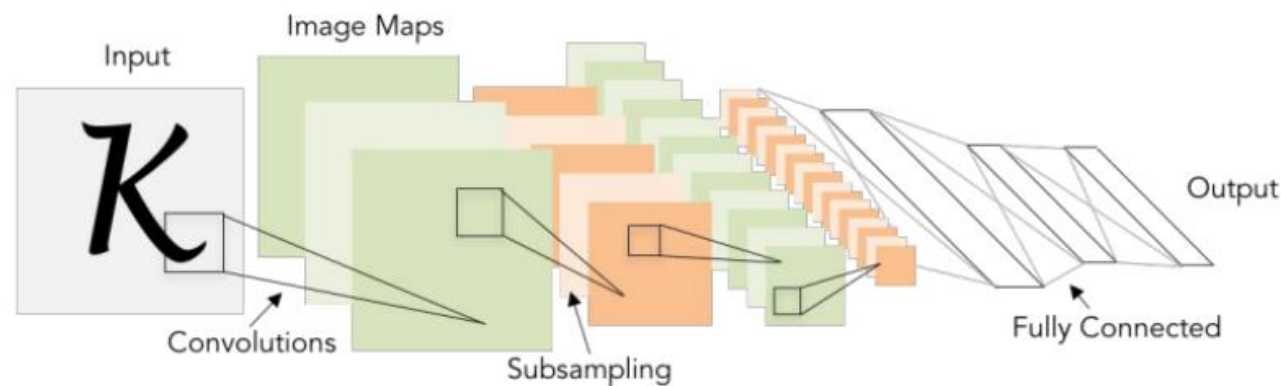
- Ý tưởng CNNs xuất phát đầu tiên từ công trình của Fukushima năm 1980

Convolutional neural network - CNN

- Lịch sử CNNs

Gradient-based learning applied to document recognition

[LeCun, Bottou, Bengio, Haffner 1998]



- Năm 1998, LeCun áp dụng BackProp huấn luyện mạng CNNs cho bài toán nhận dạng văn bản

Convolutional neural network - CNN

- Lịch sử CNNs

ImageNet Classification with Deep Convolutional Neural Networks
[Krizhevsky, Sutskever, Hinton, 2012]

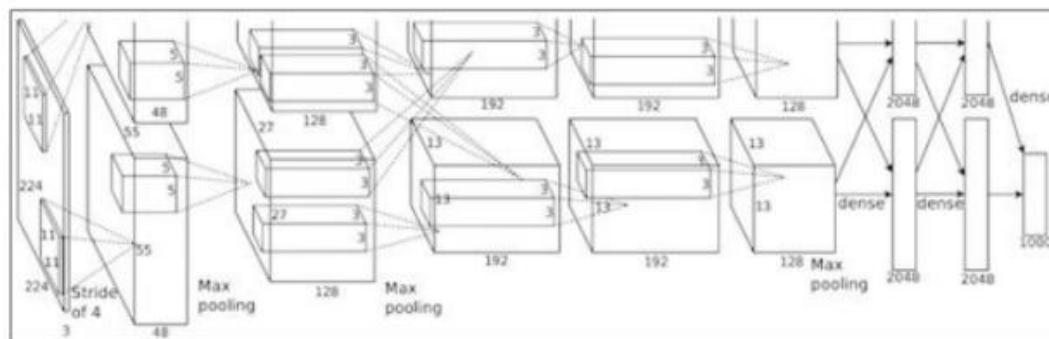


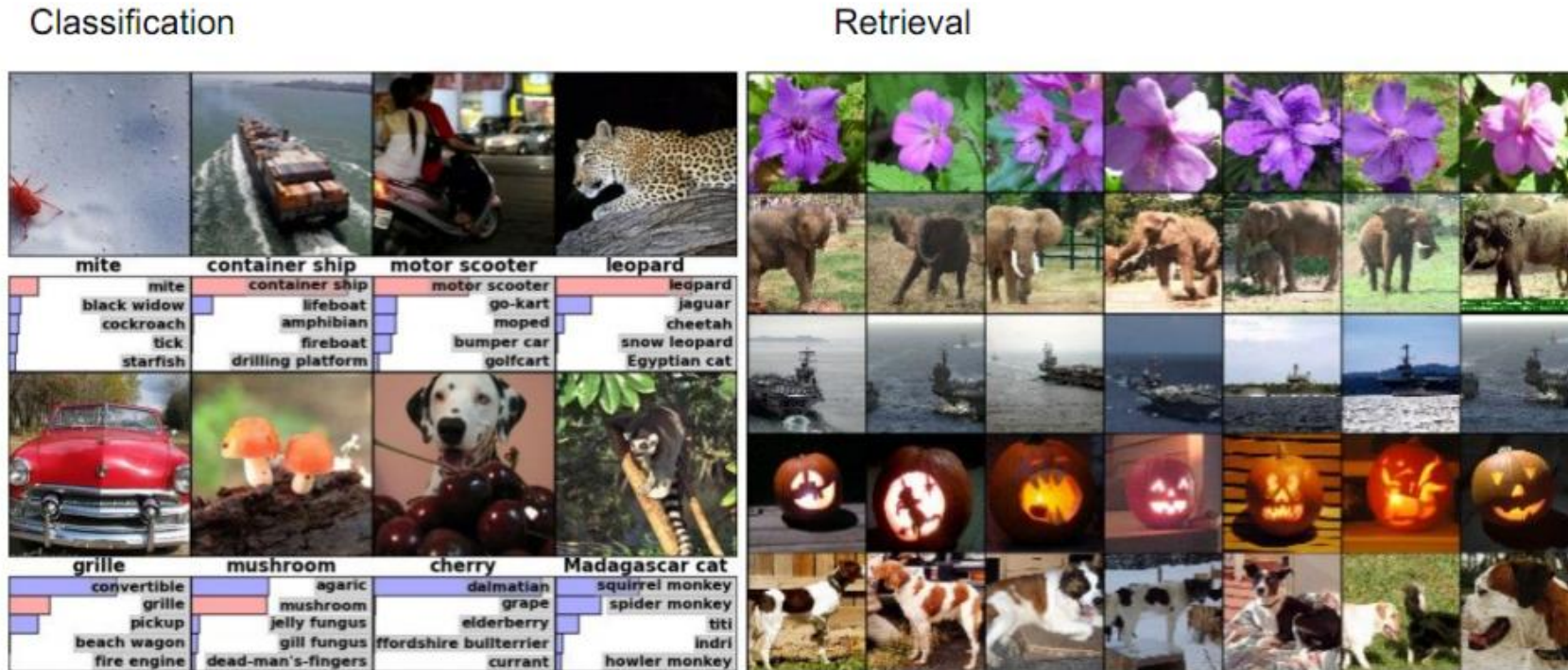
Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

“AlexNet”

- Năm 2012, CNNs gây tiếng vang lớn khi vô địch cuộc thi ILSRC 2012, vượt xa phương pháp đứng thứ 2 theo cách tiếp cận thị giác máy tính truyền thống.

Convolutional neural network - CNN

- Lịch sử CNNs



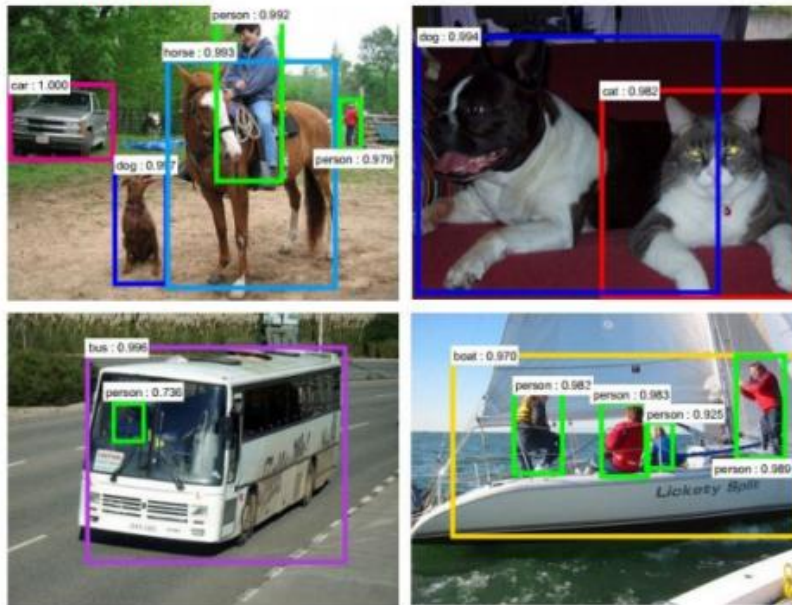
Figures copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

- Hiện nay CNNs ứng dụng khắp nơi, ví dụ trong bài toán phân loại ảnh, truy vấn ảnh

Convolutional neural network - CNN

- Lịch sử CNNs

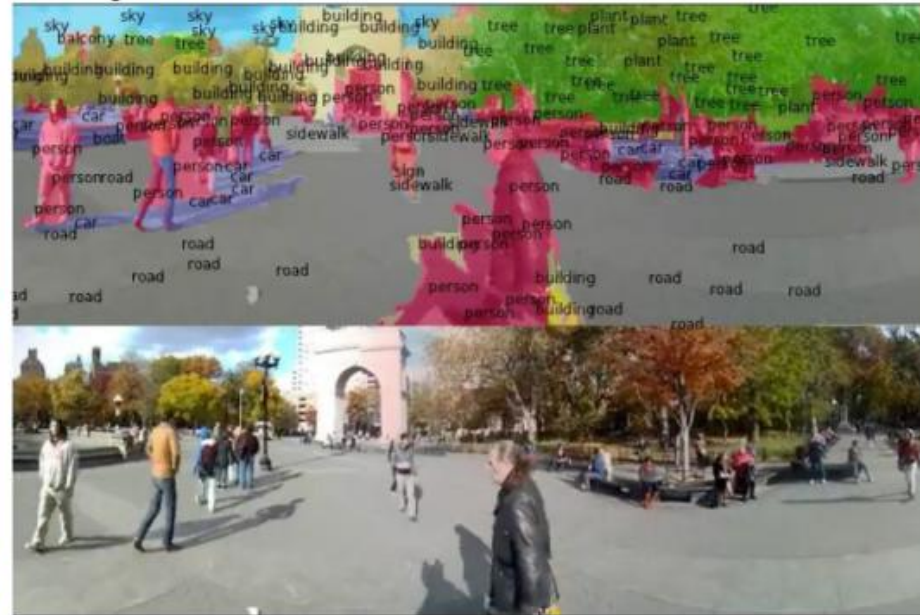
Detection



Figures copyright Shaoqing Ren, Kaiming He, Ross Girshick, Jian Sun, 2015. Reproduced with permission.

[Faster R-CNN: Ren, He, Girshick, Sun 2015]

Segmentation



Figures copyright Clement Farabet, 2012.
Reproduced with permission.

[Farabet et al., 2012]

- Ứng dụng CNNs trong bài toán phát hiện đối tượng, phân đoạn ảnh

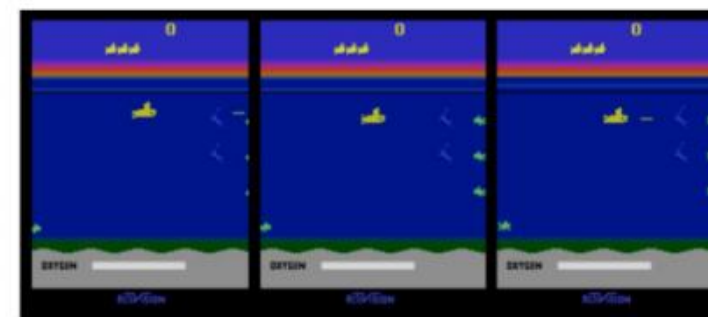
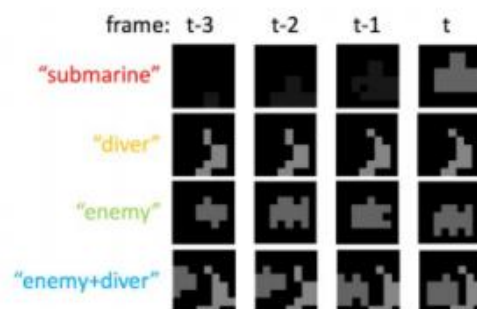
Convolutional neural network - CNN

- Lịch sử CNNs



Images are examples of pose estimation, not actually from Toshev & Szegedy 2014. Copyright Lane McIntosh.

[Toshev, Szegedy 2014]



[Guo et al. 2014]

Figures copyright Xiaoxiao Guo, Satinder Singh, Honglak Lee, Richard Lewis, and Xiaoshi Wang, 2014. Reproduced with permission.

- Ứng dụng CNNs trong nhận dạng dáng người (human pose), trong trò chơi...

Convolutional neural network - CNN

- Một số mạng CNNs cơ bản
 - **LeNet-5**
 - **AlexNet**
 - **VGG**
 - **GoogleNet**
 - **ResNet**

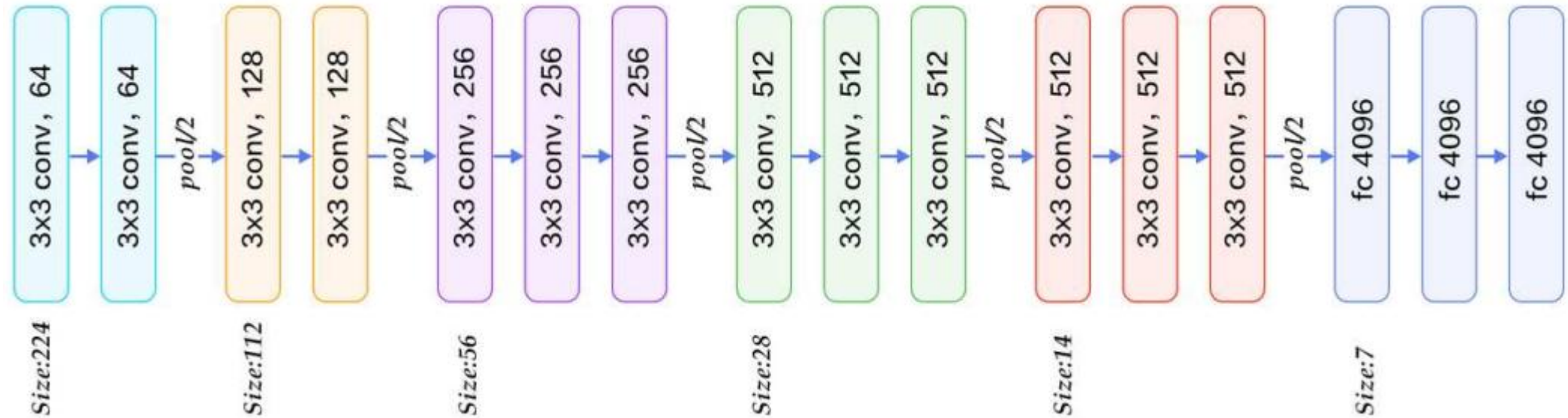
Convolutional neural network - CNN

- Mạng VGG16

- VGG16 là mạng convolutional neural network được đề xuất bởi K. Simonyan và A. Zisserman, University of Oxford.
- Model sau khi huấn luyện bởi mạng VGG16 đạt độ chính xác 92.7% top-5 test trong dữ liệu ImageNet gồm 14 triệu hình ảnh thuộc 1000 lớp khác nhau.
- Giờ ta áp dụng kiến thức ở trên để phân tích mạng VGG16.

Convolutional neural network - CNN

- Mạng VGG16



Kiến trúc vgg16 conv: convolutional layer, pool: pooling layer, fc: fully connected layer

Convolutional neural network - CNN

- Mạng VGG16

- Phân tích:

- Convolutional layer: kích thước 3×3 , padding=1, stride=1. Tại sao không ghi stride, padding mà vẫn biết? Vì mặc định sẽ là stride=1 và padding để cho output cùng width và height với input.
 - Pool/2: max pooling layer với size 2×2
 - 3×3 conv, 64: thì 64 là số kernel áp dụng trong layer đấy, hay depth của output của layer đấy.
 - Càng các convolutional layer sau thì kích thước width, height càng giảm nhưng depth càng tăng.
 - Sau khá nhiều convolutional layer và pooling layer thì dữ liệu được flatten và cho vào fully connected layer.